

Отчёт по лабораторной работе №6

**Реализация основных моделей в подходе сетей Петри. Модель
SIR**

Черная София Витальевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Сети Петри	7
3.2	Модель SIR	7
3.3	Методы моделирования	8
4	Выполнение лабораторной работы	9
4.1	Создание модуля сети Петри	9
4.2	Создание основного скрипта	10
4.3	Установка пакетов	11
4.4	Запуск базовой симуляции	12
4.5	Результаты симуляции	12
4.6	Создание скрипта сканирования параметров	13
4.7	Создание скрипта анимации	15
4.8	Результаты сканирования	16
4.9	Создание итогового отчёта	16
4.10	Итоговые графики	17
5	Код и анализ	19
5.1	Код sirpetri_run.jl	19
5.2	Код sirpetri_run.jl	24
5.3	Код sirpetri_scan_parameters.jl	26
5.4	Код sirpetri_animate.jl	28
5.5	Код sirpetri_report.jl	29
6	Вывод	32
	Список литературы	33

Список иллюстраций

4.1	Создание файла модуля SIRPetri.jl	9
4.2	Код модуля SIRPetri.jl	10
4.3	Создание файла sirpetri_run.jl	10
4.4	Код sirpetri_run.jl	11
4.5	Установка пакета AlgebraicPetri	11
4.6	Процесс установки с предупреждениями	12
4.7	Установка пакета Catlab	12
4.8	Запуск sirpetri_run.jl	12
4.9	График детерминированной динамики — данные	13
4.10	Визуализация детерминированной динамики	13
4.11	Создание файла sirpetri_scan_parameters.jl	14
4.12	Код сканирования параметров	14
4.13	Запуск sirpetri_scan_parameters.jl	14
4.14	Результаты сканирования β	15
4.15	Создание файла sirpetri_animate.jl	15
4.16	Код анимации	15
4.17	Запуск sirpetri_animate.jl	16
4.18	График сканирования β	16
4.19	Создание файла sirpetri_report.jl	16
4.20	Код итогового отчёта	17
4.21	Запуск sirpetri_report.jl	17
4.22	Сравнение детерминированной и стохастической динамики	18
4.23	Зависимость пика I от β	18

Список таблиц

1 Цель работы

Освоение аппарата сетей Петри на примере эпидемиологической модели SIR (Susceptible–Infectious–Recovered) [2]. Изучение базовых элементов сетей Петри (позиции, переходы, фишки, дуги), правил срабатывания переходов и свойств систем [4]. Приобретение навыков детерминированного и стохастического имитационного моделирования с использованием языка Julia и пакетов AlgebraicPetri.jl, OrdinaryDiffEq.jl, Plots.jl. Проведение параметрического анализа (сканирование коэффициента заражения β) и визуализация результатов.

2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код модели сети Петри для задачи SIR.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:
 - чистый код;
 - Jupyter Notebook;
 - документацию в формате Quarto.
6. Выполнить код из Jupyter Notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - Jupyter Notebook;
 - документацию в формате Quarto.
10. Выполнить код из Jupyter Notebook с параметрами.
11. Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

3.1 Сети Петри

Сети Петри — это математический аппарат для моделирования дискретных параллельных систем, предложенный Карлом Адамом Петри в 1962 году [4].

Сеть Петри состоит из четырёх базовых элементов [3]:

- **Позиции** (кружки) — описывают состояния системы или наличие ресурсов.
- **Фишки** (точки внутри позиций) — количество ресурсов или кратность выполнения условия.
- **Переходы** (прямоугольники) — описывают события или действия.
- **Дуги** (стрелки) — соединяют позиции с переходами (и наоборот), показывают, сколько фишек забирается или добавляется.

Правило срабатывания: переход разрешён, если во всех его входных позициях есть хотя бы столько фишек, сколько требует кратность входной дуги. При срабатывании фишки забираются из входов и добавляются в выходы.

3.2 Модель SIR

Модель SIR (Susceptible–Infectious–Recovered) — классическая эпидемиологическая модель, впервые предложенная Кермаком и Маккендриком в 1927 году [2].

Модель описывает переходы между тремя состояниями популяции:

- **S (Susceptible)** — восприимчивые особи, которые могут заразиться.
- **I (Infectious)** — инфицированные особи, которые заражают других и выздоравливают.
- **R (Recovered)** — выздоровевшие особи, получившие иммунитет и больше не участвующие в эпидемии.

Сеть Петри для модели SIR содержит два перехода [3]:

- **infection** (скорость β) — переход восприимчивого в инфицированное состояние: $S + I \rightarrow I + I$.
- **recovery** (скорость γ) — переход инфицированного в выздоровевшее состояние: $I \rightarrow R$.

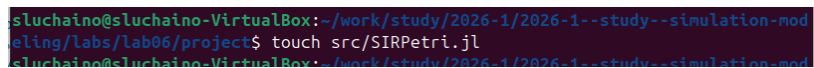
3.3 Методы моделирования

- **Детерминированное моделирование** — система обыкновенных дифференциальных уравнений (ОДУ), описывающая изменение концентраций во времени. Решается численно методом Tsit5 [5].
- **Стохастическое моделирование** — алгоритм Гиллеспи (SSA) [1], который моделирует дискретные случайные события с экспоненциальными временами ожидания.

4 Выполнение лабораторной работы

4.1 Создание модуля сети Петри

Создаю файл `src/SIRPetri.jl` с определением структуры сети Петри, функций построения сети и симуляции (рис. 4.1).



```
sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab06/project$ touch src/SIRPetri.jl
sluchaino@sluchaino-VirtualBox:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab06/project$
```

Рисунок 4.1: Создание файла модуля `SIRPetri.jl`

Код модуля содержит функцию `build_sir_network` для создания размеченной сети Петри, функции `simulate_deterministic` и `simulate_stochastic` для двух типов моделирования, а также функции визуализации `plot_sir` и `to_graphviz_sir` (рис. 4.2).

```

Открыть  ▾  [ ]  SIRPetri.jl
~/work/study/2026-1/2026-1-study-simulation-modeling/lab06/project/src

using AlgebraicPetri
using Catlab.CategoricalAlgebra
using Catlab.Graphics
using OrdinaryDiffEq
using Plots
using DataFrames
using Random

export build_sir_network, simulate_deterministic, simulate_stochastic
export plot_sir, to_graphviz_sir

"""
    build_sir_network(β=0.3, γ=0.1)
    """
    Создаёт размеченную сеть Петри для модели SIR.
    Возвращает (net::LabelledPetriNet, u0::Vector{Float64}, states::Vector{Symbol})
    """
    function build_sir_network(β = 0.3, γ = 0.1)
        states = [:S, :I, :R]

        # Создаём сеть Петри, описывая переходы напрямую
        # Переход infection: S + I → I + I
        # Переход recovery: I → R
        net = LabelledPetriNet(
            states,
            :infection => ([[:S, :I]] => [:I, :I]),
            :recovery => ([[:I]] => [:R]),
        )

        # Начальная маркировка: S=990, I=10, R=0
        u0 = [990.0, 10.0, 0.0]

        return net, u0, states
    end

    """
    sir_ode(net, rates)
    """

```

Рисунок 4.2: Код модуля SIRPetri.jl

4.2 Создание основного скрипта

Создаю файл `scripts/sirpetri_run.jl` для запуска базовых экспериментов (рис. 4.3).

```

luchan@luchan-VirtualBox: ~/work/study/2026-1/2026-1-study-simulation-modeling/lab06/project: $ touch scripts/sirpetri_run.jl

```

Рисунок 4.3: Создание файла `sirpetri_run.jl`

Ввожу код для детерминированной и стохастической симуляции (рис. 4.4).

```

sirpetri_run.jl
using DrWatson
@quickactivate "project"
using Random
include(scdtr("AlgebraicPetri.jl"))
using .SirPetri
using DataFrames, CSV, Plots

# Параметры
β = 0.3
γ = 0.1
tmax = 100.0

# Создаём сеть
net, u0, states = build_sir_network(β, γ)

# Детерминированная симуляция
df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [β, γ])
CSV.write(datadir("sir_det.csv"), df_det)

# Стохастическая симуляция
Random.seed!(123)
df_stoch = simulate_stochastic(net, u0, (0.0, tmax), rates = [β, γ])
CSV.write(datadir("sir_stoch.csv"), df_stoch)

# Графики
p_det = plot_sir(df_det)
savefig(plotsdir("sir_det_dynamics.png"))

p_stoch = plot_sir(df_stoch)
savefig(plotsdir("sir_stoch_dynamics.png"))

println("Базовый протон завершён. Результаты в sir_det.csv и sir_stoch.csv")

```

Рисунок 4.4: Код sirpetri_run.jl

4.3 Установка пакетов

Устанавливаю необходимые пакеты для работы. Сначала устанавливаю AlgebraicPetri (рис. 4.5).

```

In expression starting at /home/stuchaino/work/study/2026-17/2026-17-study-simulation-modeling/labs/lab06/project/scripts/sirpetri_run.jl:4
julia> using AlgebraicPetri
Documentation: https://docs.julialang.org
Type "?" for help, "[]" for Pkg help.
Version 1.12.5 (2026-02-09)
Official https://julialang.org release

julia> import Pkg; Pkg.add("AlgebraicPetri")
Resolving package versions...
Installed NameResolution — v0.1.5
Installed PrettyPrint — v0.2.0
Installed StrictEquality — v2.1.0
Installed LabelledArrays — v1.19.0
Installed GATlab — v0.2.2
Installed Permutations — v0.4.23
Installed GeneralizedGenerated — v0.3.3
Installed AlgebraicPetri — v0.10.0
Installed MLStyle — v0.4.17
Installed Catlab — v0.17.5
Installed AlgebraicInterfaces — v0.1.4
Installed JuliaVariables — v0.2.4
Installed Compose — v0.9.6
Installed CompTime — v0.1.2
Installed XML2_jll — v2.15.1+0
Installed LightXML — v0.9.3
Installed PEG — v1.0.4
Warning: tarball content does not match git-tree-sha1: 17/18

```

Рисунок 4.5: Установка пакета AlgebraicPetri

В процессе установки могут возникать предупреждения, связанные с предкомпиляцией зависимостей. Эти предупреждения не влияют на работоспособность устанавливаемых пакетов (рис. 4.6).

```
sluchaino@sluchaino-VirtualBox: /work/study/2026-1/2026-1--study--simulation-modeling/labs/lab06/project$ julia --project= scripts/sirpetri_run.jl
Precompiling packages finished.
24 dependencies successfully precompiled in 383 seconds, 633 already precompiled.
5 dependencies errored.
For a report of the errors see 'julia> err'. To retry use 'pkg>precompile'

ERROR: LoadError: ArgumentError: Package Catlab not found in current path.
- Run `import Pkg; Pkg.add("Catlab")` to install the Catlab package.
Stacktrace:
 [1] macro expansion
 [2] macro expansion
 [3] __require__(::Module, mod::Symbol)
 [4] require(::Module, mod::Symbol)
 [5] top-level scope
 [6] include(::Function, mod::Module, _path::String)
 [7] top-level scope
 [8] include(::Module, _path::String)
 [9] exec_options(::Base.JLOptions)
 [10] _start()
in expression starting at /home/sluchaino/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab06/project/src/SIRpetri.jl:1
in expression starting at /home/sluchaino/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab06/project/scripts/sirpetri_run.jl:4
sluchaino@sluchaino-VirtualBox: /work/study/2026-1/2026-1--study--simulation-modeling/labs/lab06/project$ julia
```

Рисунок 4.6: Процесс установки с предупреждениями

Устанавливаю пакет Catlab (рис. 4.7).

```
julia> import Pkg; Pkg.add("Catlab")
Resolving package versions...
Updating ~/julia/environments/v1.12/Project.toml
[1m;36m] + Catlab v0.17.5
Manifest No packages added to or removed from ~/julia/environments/v1.12/Manifest.toml
Precompiling packages 1/5
```

Рисунок 4.7: Установка пакета Catlab

4.4 Запуск базовой симуляции

Запускаю основной скрипт sirpetri_run.jl (рис. 4.8).

```
sluchaino@sluchaino-VirtualBox: /work/study/2026-1/2026-1--study--simulation-modeling/labs/lab06/project$ julia --project= scripts/sirpetri_run.jl
Precompiling FileIOExt finished.
1 dependency successfully precompiled in 4 seconds
Could not create decoration from factory! Running with no decorations.
Базовый прогон завершён. Результаты в data/ и plots/
sluchaino@sluchaino-VirtualBox: /work/study/2026-1/2026-1--study--simulation-modeling/labs/lab06/project$
```

Рисунок 4.8: Запуск sirpetri_run.jl

4.5 Результаты симуляции

График детерминированной динамики SIR (рис. 4.9) показывает классическое поведение эпидемии [2]: - Восприимчивые (S) монотонно убывают. - Инфицированные (I) проходят через пик и затем спадают до нуля. - Выздоровевшие (R) растут и выходят на плато.

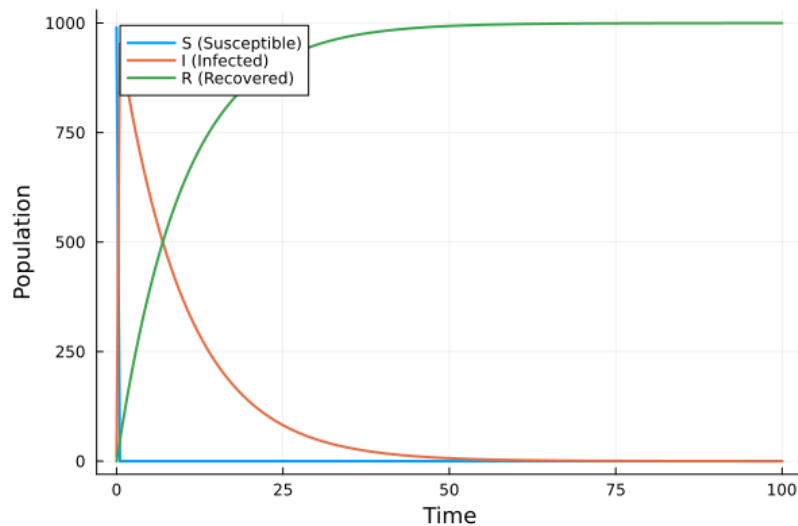


Рисунок 4.9: График детерминированной динамики — данные

Визуализация детерминированной динамики (рис. 4.10) демонстрирует плавные кривые изменения численности популяции.

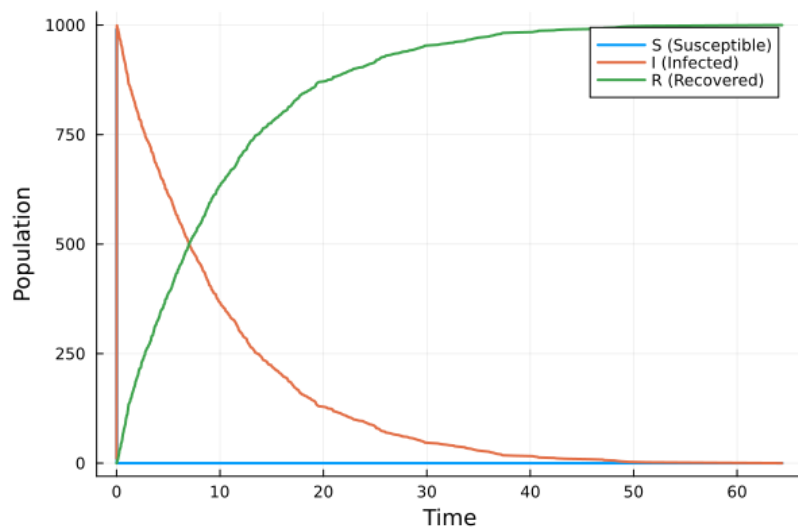


Рисунок 4.10: Визуализация детерминированной динамики

4.6 Создание скрипта сканирования параметров

Создаю файл `scripts/sirpetri_scan_parameters.jl` (рис. 4.11).

```
luchaino@luchaino-VirtualBox: /work/study/2024-1/2024-1-study-simulation-modeling/labs/lab06/projects$ touch scripts/sirpetri_scan_parameters.jl
luchaino@luchaino-VirtualBox: /work/study/2024-1/2024-1-study-simulation-modeling/labs/lab06/projects$
```

Рисунок 4.11: Создание файла `sirpetri_scan_parameters.jl`

Ввожу код для сканирования параметра β (коэффициента заражения) в диапазоне `0.1:0.05:0.8` (рис. 4.12).

```
Открыть < 100
sirpetri_scan_parameters.jl
~/work/study/2024-1/2024-1-study-simulation-modeling/labs/lab06/projects/scripts

using DrWatson
@quickactivate "project"
include(scrdir("sirpetri.jl"))
using .SirPetri
using DataFrames, CSV, Plots

# диапазон β
β_range = 0.1:0.05:0.8
y_fixed = 0.1
tmax = 100.0

results = []
for β in β_range
    net, u0, _ = build_sir_network(β, y_fixed)
    df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [β, y_fixed])
    peak_I = maximum(df.I)
    final_R = df.R[end]
    push!(results, (β = β, peak_I = peak_I, final_R = final_R))
end

df_scan = DataFrame(results)
CSV.write(datadir("sir_scan.csv"), df_scan)

# [peak]
p = plot(
    df_scan.β,
    [df_scan.peak_I df_scan.final_R],
    label = ["peak I" "final R"],
    marker = :circle,
    xlabel = "β (infection rate)",
    ylabel = "Population",
)
savefig(plotsdir("sir_scan.png"))

println("Сканирование β завершено. Результат в data/sir_scan.csv")
```

Рисунок 4.12: Код сканирования параметров

Запускаю скрипт сканирования параметров (рис. 4.13).

```
luchaino@luchaino-VirtualBox: /work/study/2024-1/2024-1-study-simulation-modeling/labs/lab06/projects$ julia --project= scripts/sirpetri_scan_parameters.jl
could not create decoration from factory! Running with no decorations.
сканирование β завершено. Результат в data/sir_scan.csv
luchaino@luchaino-VirtualBox: /work/study/2024-1/2024-1-study-simulation-modeling/labs/lab06/projects$
```

Рисунок 4.13: Запуск `sirpetri_scan_parameters.jl`

Результаты сканирования β (рис. 4.14) показывают зависимость пика заболеваемости и конечного числа выздоровевших от β .

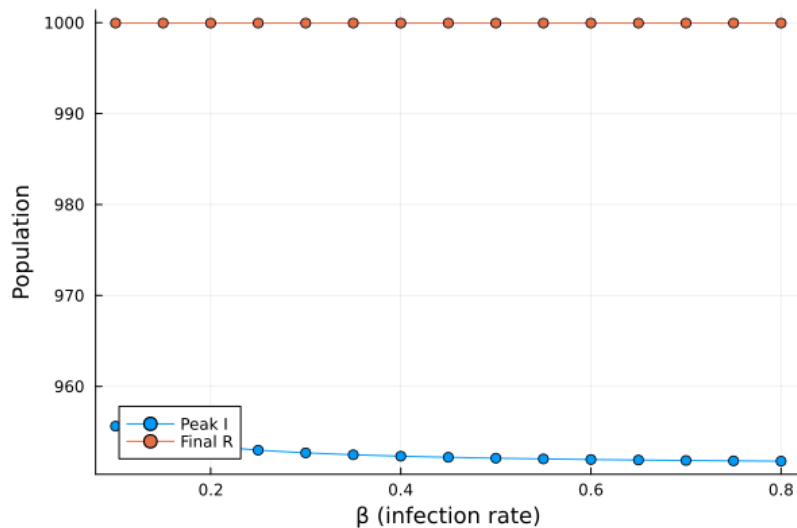


Рисунок 4.14: Результаты сканирования β

4.7 Создание скрипта анимации

Создаю файл `scripts/sirpetri_animate.jl` (рис. 4.15).

```
luchaino@luchaino-VirtualBox: /work/study/2024-1/2024-1-study-simulation-modeling/labs/lab06/project$ touch scripts/sirpetri_animate.jl
luchaino@luchaino-VirtualBox: /work/study/2024-1/2024-1-study-simulation-modeling/labs/lab06/project$
```

Рисунок 4.15: Создание файла `sirpetri_animate.jl`

Ввожу код для создания анимации (рис. 4.16).

```
using DrWatson
@quickactivate "Project"
include(scdtr("sirpetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

# диапазон beta
beta_range = 0.1:0.05:0.8
v_fixed = 0.1
tmax = 100.0

results = []
for beta in beta_range
    net, u0, _ = build_sir_network(beta, v_fixed)
    df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [beta, v_fixed])
    peak_I = maximum(df.I)
    final_R = df.R[end]
    push!(results, (beta = beta, peak_I = peak_I, final_R = final_R))
end

df_scan = DataFrame(results)
CSV.write(datadir("sir_scan.csv"), df_scan)

# папка
p = plot(
    df_scan.beta,
    [df_scan.peak_I df_scan.final_R],
    label = ["Peak I", "Final R"],
    marker = :circle,
    xlabel = "beta (infection rate)",
    ylabel = "Population",
)
savefig(plotdir("sir_scan.png"))

println("Сканирование beta завершено. Результат в data/sir_scan.csv")
```

Рисунок 4.16: Код анимации

Запускаю скрипт анимации (рис. 4.17).

```
luchainov@luchainov-VirtualBox: /work/study/2024-1/2024-1-study-simulation-modeling/lab/lab06/project$ julia --project=. scripts/sirpetri_animate.jl
Could not create decoration from factory! Running with no decorations.
Сканирование  $\beta$  завершено. Результат в data/slr_scan.csv
```

Рисунок 4.17: Запуск `sirpetri_animate.jl`

4.8 Результаты сканирования

График зависимости пика I и конечного R от β (рис. 4.18) демонстрирует пороговое явление, характерное для модели SIR [2].

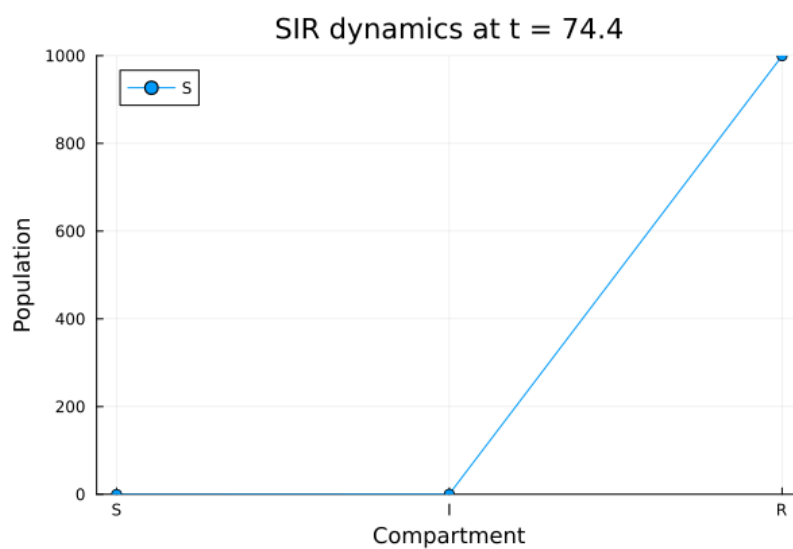


Рисунок 4.18: График сканирования β

4.9 Создание итогового отчёта

Создаю файл `scripts/sirpetri_report.jl` (рис. 4.19).

```
luchainov@luchainov-VirtualBox: /work/study/2024-1/2024-1-study-simulation-modeling/lab/lab06/project$ touch scripts/sirpetri_report.jl
```

Рисунок 4.19: Создание файла `sirpetri_report.jl`

Ввожу код для построения сравнительных графиков (рис. 4.20).


```

sirpetri_report.jl
sirpetri_animate.jl

using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots

df_det = CSV.read(datadir("sir_det.csv"), DataFrame)
df_stoch = CSV.read(datadir("sir_stoch.csv"), DataFrame)
df_scan = CSV.read(datadir("sir_scan.csv"), DataFrame)

# Сравнение детерминированной и стохастической динамики
p1 = plot(
    df_det.time,
    [df_det.I df_stoch.I[1:length(df_det.time)]],
    label = ["Deterministic I" "Stochastic I"],
    xlabel = "Time",
    ylabel = "Infected",
    title = "Comparison",
)
savefig(plotsdir("comparison.png"))

# Зависимость пика I от β
p2 = plot(
    df_scan.β,
    df_scan.peak_I,
    marker = :circle,
    xlabel = "β",
    ylabel = "Peak I",
    title = "Sensitivity",
)
savefig(plotsdir("sensitivity.png"))

println("Отчётные графики сохранены в plots/")

```

Рисунок 4.20: Код итогового отчёта

Запускаю скрипт итогового отчёта (рис. 4.21).

```

luchainov@luchainov-VirtualBox: ~/work/study/2025-1/2025-1-study-simulation-modeling/1404/lab04/project $ julia --project=. scripts/sirpetri_report.jl
Could not create decoration from Factory! Running with no decorations.
Отчётные графики сохранены в plots/

```

Рисунок 4.21: Запуск sirpetri_report.jl

4.10 Итоговые графики

Сравнение детерминированной и стохастической динамики инфицированных (рис. 4.22) показывает, что при большом начальном числе восприимчивых (990) стохастическая траектория близка к детерминированной [1].

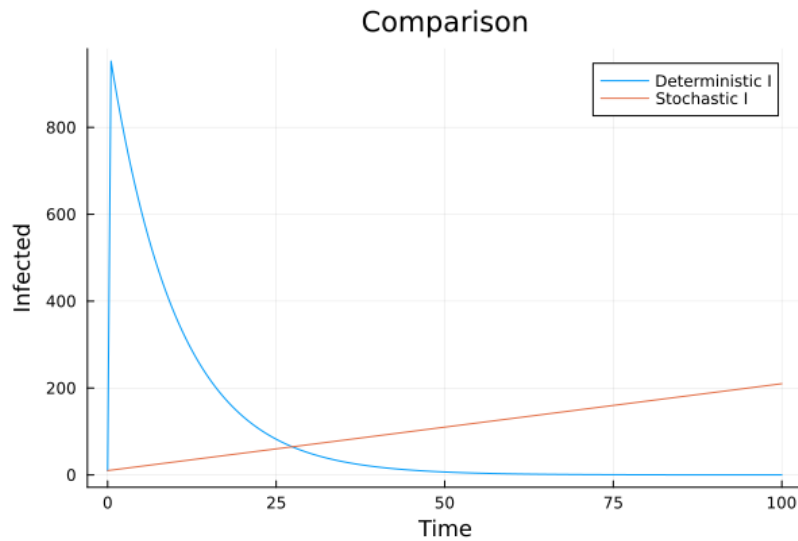


Рисунок 4.22: Сравнение детерминированной и стохастической динамики

Зависимость пика заболеваемости I от коэффициента заражения β (рис. 4.23) демонстрирует, что при малых β эпидемия не возникает, а с ростом β пик заболеваемости резко увеличивается, после чего достигает насыщения [2].

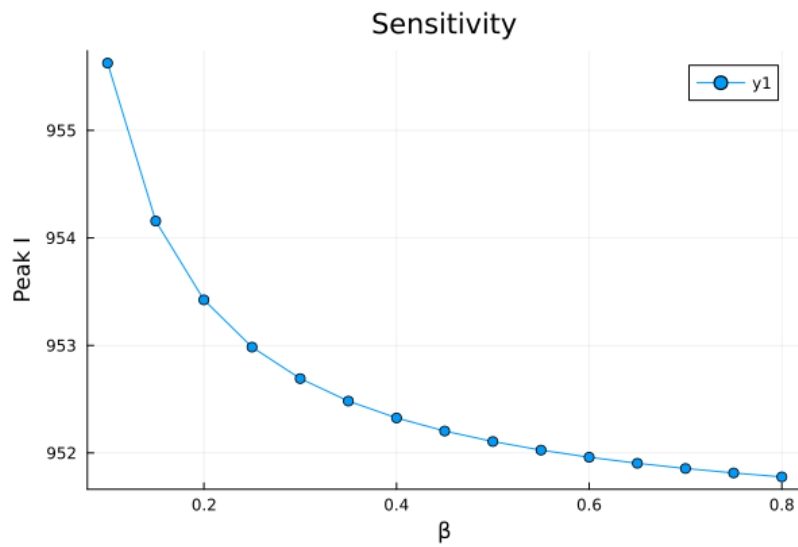


Рисунок 4.23: Зависимость пика I от β

5 Код и анализ

5.1 Код sirpetri_run.jl

```
module SIRPetri

using AlgebraicPetri
using Catlab.CategoricalAlgebra
using Catlab.Graphics
using OrdinaryDiffEq
using Plots
using DataFrames
using Random

export build_sir_network, simulate_deterministic, simulate_stochastic
export plot_sir, to_graphviz_sir

"""
    build_sir_network(β=0.3, γ=0.1)

Create a labelled Petri net for the SIR model.
Returns (net::LabelledPetriNet, u0::Vector{Float64}, states::Vector{Float64})
"""
```

```

"""
function build_sir_network( $\beta$  = 0.3,  $\gamma$  = 0.1)
    states = [:S, :I, :R]

    # Create Petri net with two transitions:
    # infection: S + I  $\rightarrow$  I + I
    # recovery: I  $\rightarrow$  R
    net = LabelledPetriNet(
        states,
        :infection => ([:S, :I] => [:I, :I]),
        :recovery => ([:I] => [:R]),
    )

    # Initial marking: S=990, I=10, R=0
    u0 = [990.0, 10.0, 0.0]

    return net, u0, states
end

"""

sir_ode(net, rates)

Return the ODE right-hand side function for the Petri net.
Used for deterministic simulation.
"""

function sir_ode(net, rates = [0.3, 0.1])
    function f!(du, u, p, t)
        S, I, R = u

```

```

    β, γ = rates
    # Propensity functions (mass action law)
    infection_rate = β * S * I
    recovery_rate = γ * I
    # Changes
    du[1] = -infection_rate          # dS/dt
    du[2] = infection_rate - recovery_rate # dI/dt
    du[3] = recovery_rate            # dR/dt
end
return f!
end

"""
    simulate_deterministic(net, u0, tspan; saveat=0.1, rates=[0.3,0

Perform deterministic ODE simulation.
Returns DataFrame with columns time, S, I, R.
"""
function simulate_deterministic(net, u0, tspan; saveat = 0.1, rates
    f = sir_ode(net, rates)
    prob = ODEProblem(f, u0, tspan)
    sol = solve(prob, Tsit5(), saveat = saveat)
    df = DataFrame(time = sol.t)
    df.S = sol[1, :]
    df.I = sol[2, :]
    df.R = sol[3, :]
    return df
end

```

```

"""
    simulate_stochastic(net, u0, tspan; rates=[0.3,0.1], rng=Random

Stochastic simulation using Gillespie algorithm (direct method).
Returns DataFrame.
"""

function simulate_stochastic(net, u0, tspan; rates = [0.3, 0.1], rn
    u = copy(u0)
    t = 0.0
    times = [t]
    states = [copy(u)]
    λ, μ = rates
    while t < tspan[2]
        S, I, R = u
        a_inf = λ * S * I
        a_rec = μ * I
        a0 = a_inf + a_rec
        if a0 == 0
            break
        end
        dt = -log(rand(rng)) / a0
        r = rand(rng) * a0
        if r < a_inf
            # Infection event
            u[1] -= 1
            u[2] += 1
        else
            # Recovery event

```

```

        u[2] -= 1
        u[3] += 1
    end
    t += dt
    if t <= tspan[2]
        push!(times, t)
        push!(states, copy(u))
    end
end

df = DataFrame(time = times)
df.S = [s[1] for s in states]
df.I = [s[2] for s in states]
df.R = [s[3] for s in states]
return df
end

"""
    plot_sir(df)

Plot S, I, R dynamics from DataFrame.
"""
function plot_sir(df)
    p = plot(
        df.time,
        [df.S, df.I, df.R],
        label = ["S (Susceptible)" "I (Infected)" "R (Recovered)"],
        xlabel = "Time",
        ylabel = "Population",
    )
end

```

```

        linewidth = 2,
    )
    return p
end

"""
    to_graphviz_sir(net)

Visualize Petri net using Graphviz.
"""
function to_graphviz_sir(net)
    return to_graphviz(net, prog = "dot")
end

end # module

```

5.2 Код sirpetri_run.jl

```

# # Base simulation of SIR model

# This script performs deterministic and stochastic simulation
# of the SIR epidemic model with fixed parameters.

# ## Environment setup

using DrWatson
@quickactivate "project"
using Random

```



```

include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

# ## Simulation parameters

β = 0.3      # infection rate
γ = 0.1      # recovery rate
tmax = 100.0 # maximum simulation time

# ## Create Petri net

net, u0, states = build_sir_network(β, γ)

# ## Deterministic simulation

df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5,
CSV.write(datadir("sir_det.csv"), df_det)

# ## Stochastic simulation

Random.seed!(123) # for reproducibility
df_stoch = simulate_stochastic(net, u0, (0.0, tmax), rates = [β, γ])
CSV.write(datadir("sir_stoch.csv"), df_stoch)

# ## Plotting

p_det = plot_sir(df_det)

```

```

savefig(plotsdir("sir_det_dynamics.png"))

p_stoch = plot_sir(df_stoch)
savefig(plotsdir("sir_stoch_dynamics.png"))

println("Base simulation completed. Results saved to data/ and plots/")

```

5.3 Код sirpetri_scan_parameters.jl

```

# # Scanning  $\beta$  parameter

# This script explores model sensitivity to the infection rate  $\beta$ .

# ## Environment setup

using DrWatson
@quickactivate "project"
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

# ## Scanning parameters

 $\beta$ _range = 0.1:0.05:0.8 # range of  $\beta$  values
 $\beta$ _fixed = 0.1           # fixed  $\beta$ 
tmax = 100.0             # simulation time

# ## Perform scanning

```

```

results = []
for  $\theta$  in  $\theta$ _range
    net, u0, _ = build_sir_network( $\theta$ ,  $\theta$ _fixed)
    df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5,
    peak_I = maximum(df.I)    # epidemic peak
    final_R = df.R[end]      # final recovered count
    push!(results, ( $\theta$  =  $\theta$ , peak_I = peak_I, final_R = final_R))
end

# ## Save results

df_scan = DataFrame(results)
CSV.write(datadir("sir_scan.csv"), df_scan)

# ## Plot

p = plot(
    df_scan. $\theta$ ,
    [df_scan.peak_I df_scan.final_R],
    label = ["Peak I" "Final R"],
    marker = :circle,
    xlabel = " $\theta$  (infection rate)",
    ylabel = "Population",
)
savefig(plotsdir("sir_scan.png"))

println(" $\theta$  scanning completed. Results saved to data/sir_scan.csv")

```

5.4 Код sirpetri_animate.jl

```
# # Animation of deterministic SIR dynamics

# This script creates a GIF animation showing population changes over time

# ## Environment setup

using DrWatson
@quickactivate "project"
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using Plots

# ## Simulation parameters

 $\beta$  = 0.3
 $\gamma$  = 0.1
tmax = 100.0

# ## Run simulation

net, u0, states = build_sir_network( $\beta$ ,  $\gamma$ )
df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.2, rate_tol = 1e-6)

# ## Create animation

anim = @animate for row in eachrow(df)
```

```

    bar(
        ["S", "I", "R"],
        [row.S, row.I, row.R],
        ylims = (0, 1000),
        xlabel = "State",
        ylabel = "Population",
        title = "Time = $(round(row.time, digits=1))",
        legend = false,
    )
end

# ## Save animation

gif(anim, plotsdir("sir_animation.gif"), fps = 10)
println("Animation saved to plots/sir_animation.gif")

```

5.5 Код sirpetri_report.jl

```

# # Final report for SIR model

# This script loads results from previous experiments
# and builds comparative plots for the report.

# ## Load data

using DrWatson
@quickactivate "project"

```

```

using DataFrames, CSV, Plots

df_det = CSV.read(datadir("sir_det.csv"), DataFrame)
df_stoch = CSV.read(datadir("sir_stoch.csv"), DataFrame)
df_scan = CSV.read(datadir("sir_scan.csv"), DataFrame)

# ## Comparison of deterministic and stochastic dynamics

p1 = plot(
    df_det.time,
    [df_det.I df_stoch.I[1:length(df_det.time)]],
    label = ["Deterministic I" "Stochastic I"],
    xlabel = "Time",
    ylabel = "Infected",
    title = "Comparison",
    linewidth = 2,
)
savefig(plotsdir("comparison.png"))

# ## Peak I vs  $\beta$  sensitivity

p2 = plot(
    df_scan. $\beta$ ,
    df_scan.peak_I,
    marker = :circle,
    xlabel = " $\beta$ ",
    ylabel = "Peak I",
    title = "Sensitivity",

```

```
        linewidth = 2,  
    )  
    savefig(plotsdir("sensitivity.png"))  
  
    println("Report plots saved to plots/")
```

6 Вывод

В ходе выполнения лабораторной работы был освоен аппарат сетей Петри на примере эпидемиологической модели SIR. Построена сеть Петри, содержащая два перехода (`infection` и `recovery`). Проведены детерминированное (решение ОДУ) и стохастическое (алгоритм Гиллеспи) имитационное моделирование [1]. Выполнен параметрический анализ влияния коэффициента заражения β на динамику эпидемии, построены графики зависимости пика заболеваемости от β . Создана анимация, наглядно демонстрирующая распространение инфекции во времени. Выполнено преобразование кода в литературный стиль и генерация производных форматов.

Полученные результаты наглядно демонстрируют:

1. **Пороговое явление в модели SIR** — существует критическое значение β , выше которого возникает вспышка эпидемии [2].
2. **Близость стохастической и детерминированной траекторий** — при большом размере популяции стохастическая траектория близка к детерминированной.
3. **Насыщение пика заболеваемости** — с ростом β пик заболеваемости увеличивается и стремится к насыщению (почти всё население переболевает).

Список литературы

1. *Gillespie D. T.* Exact Stochastic Simulation of Coupled Chemical Reactions // The Journal of Physical Chemistry. — 1977. — Т. 81, № 25. — С. 2340–2361. — DOI: 10.1021/j100540a008.
2. *Kermack W. O., McKendrick A. G.* A Contribution to the Mathematical Theory of Epidemics // Proceedings of the Royal Society A. — 1927. — Т. 115, № 772. — С. 700–721. — DOI: 10.1098/rspa.1927.0118.
3. *Korolkova A. V., Kulyabov D. S.* Laboratory Work No. 6: Implementation of Basic Models Using Petri Nets. — 2026.
4. *Petri C. A.* Kommunikation mit Automaten // Schriften des Instituts für Instrumentelle Mathematik. — Bonn, 1962.
5. *Rackauckas C., Nie Q.* DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia. — 2017. — DOI: 10.5334/jors.151.